

Rapport de mémoire présenté en vue d'obtenir

UE NFE 204 « Bases de Données Avancées 1 »

Projet « VOLDEMORT » : Une implémentation libre de « SimpleDB »

BLONDEAU Raphaël

Spécialité : Systèmes d'information

Enseignant : P. RIGAUX

Table des matières

Table des figures.....	3
Liste des acronymes	3
Introduction.....	4
1 Passage du modele relationnel au Nosql.....	5
1.1 L'avènement du NoSQL	5
1.2 L'approche d'AMAZON.....	11
1.3 La base DYNAMO.....	11
2 Le projet voldemort	13
2.1 Historique	13
2.2 VOLDEMORT en détails.....	14
2.3 Les principes mis en œuvre	15
2.3.1 Une architecture en couches :.....	15
2.3.2 La disponibilité.....	16
2.3.3 La distribution, Consistent Haching.....	17
2.3.4 Structure de données et sérialisation	17
2.3.5 La réplication et la consistance	18
2.3.6 Les versions de données	20
2.4 Simplicité d'installation et de configuration	20
3 Discussion sur l'approche NoSQL	23
Conclusion	24
Glossaire.....	25
Webographie.....	26

Table des figures

Figure 1 : SGBD Relationnels VS Bases NoSQL (http://blog.inovia-conseil.fr).....	5
Figure 2 : Schématisation Théorème CAP (http://blog.beany.co.kr/archives/275)	6
Figure 3 : Modèle Clé/Valeur (http://blog.xebia.fr)	7
Figure 4 : Modèle Clé/Valeur Distribué (http://blog.inovia-conseil.fr)	7
Figure 5 : Modèle orienté Colonnes (http://blog.xebia.fr)	8
Figure 6 : Colonnes dynamiques (http://blog.xebia.fr)	8
Figure 7 : Modèle orienté Document (http://blog.inovia-conseil.fr).....	8
Figure 8 : Modèle orienté Document, représentation (http://blog.xebia.fr).....	9
Figure 9 : Modèle Orienté Graphe (http://blog.inovia-conseil.fr).....	9
Figure 10 : Représentation Modèle orienté Graphe (http://blog.xebia.fr)	10
Figure 11 : Représentation sociale, modèle Graphe (http://blog.xebia.fr).....	10
Figure 12 : Architecture AMAZON et DYNAMO (Article Amazon)	12
Figure 13 : LinkedIn page contacts (LinkedIn).....	13
Figure 14 : Architecture VOLDEMORT (http://www.project-voldemort.com/voldemort/).....	15
Figure 15 : Architecture Physique VOLDEMORT (http://www.project-voldemort.com/voldemort/)	15
Figure 16 : Consistent Haching (http://blog.xebia.fr/).....	17
Figure 17 : Types de données pour schéma.....	18
Figure 18 : Cluster configuration (http://blog.octo.com)	19
Figure 19 : Identification du Noeud.....	20
Figure 20 : Définition du cluster.....	21
Figure 21 : Réplication, cohérence et structure	22

Liste des acronymes

SGBD (SGBDR) : Système de Gestion de Bases de Données (Relationnel),

API (Application Programming Interface) : Interface de programmation d'applications,

PHP : Hypertext Preprocessor ; langage de programmation libre4 principalement utilisé pour produire des pages Web dynamiques via un serveur http

JSON : JavaScript Object Notation ; Format de données textuelles, générique, dérivé de la notation des objets du langage ECMAScript

SQL : Structured Query Language, (langage de requête structurée) ; Langage informatique normalisé servant à exploiter des bases de données relationnelles

XML : Extensible Markup Language (langage de balisage extensible) ; Langage informatique de balisage générique qui dérive du SGML

JAVA : langage de programmation informatique orienté objet

WEB : Désigne le World Wide Web (WWW), « toile (d'araignée) mondiale » ; Système hypertexte public fonctionnant sur Internet

TCP/IP : TCP (Transmission Control Protocol) et IP (Internet Protocol) ; Protocoles utilisés pour le transfert des données sur Internet.

CMS/SGC : Content Management System ; Famille de logiciels destinés à la conception et à la mise à jour dynamique de sites Web ou d'applications multimédia.

INTRODUCTION

Les Systèmes de Gestion de Bases de Données (SGBD) relationnels sont très implantés dans les entreprises. Ils répondent globalement à leurs besoins fonctionnels de gestion courante.

Toutefois dans le cadre d'activités particulière telles les sites web, le commerce en ligne, et plus globalement tout le secteur d'activité développé autour de l'internet ; Les entreprises doivent fournir des services à des millions de clients répartis dans le monde entier.

Les SGBD classiques axés sur le modèle relationnel atteignent leurs limites en répartition de charge pour faire face à des volumes de données très importants.

Ces données doivent pouvoir être accessible de n'importe où dans le monde, il faut alors les distribuer et les répliquer sur de très nombreux serveurs physiques. Ce qui pour les SGBD relationnel induit une très grande complexité et des coûts de mise en œuvre très élevés.

Ce constat a amené des grands acteurs de l'Internet comme AMAZON ou LinkedIn à appréhender les bases de données de manière différente. Ils ont cherché à reprendre un système simple à mettre en œuvre et connu depuis les années 1970 : Le stockage d'information associatif par clé/valeur. Ce système est notamment utilisé dans les structures de données de langages comme le PERL ou plus récemment le PHP.

Les acteurs de ces segments de marché passent du modèle relationnel au NO SQL en sacrifiant les éléments qui font la force du modèle relationnel, au profit de la simplicité du passage à l'échelle et de la performance dans le but de fournir des services spécialisés.

Dans le présent exposé je présenterais les enjeux et les principes du système NO SQL en abordant l'approche DYNAMO d'AMAZON. Je développerais ensuite le projet Voldemort, sa structure et sa mise en œuvre. Je conclurais sur les systèmes NO SQL dans le monde de l'entreprise aujourd'hui.

1 PASSAGE DU MODELE RELATIONNEL AU NOSQL

1.1 L'avènement du NoSQL

Les Systèmes de Gestion de Bases de Données (SGBD) non relationnels sont anciens. Ils sont classiquement utilisés sur les gros systèmes et dans les logiciels d'annuaires comme « LDAP ». Les Systèmes non relationnels sont généralement utilisés pour des systèmes ayant besoin de performances dans le cas où il y a beaucoup plus de lectures de données que d'écritures.

Le Terme « NOSQL » pour « Not Only SQL » a été utilisé dès 1985 signifie littéralement « Pas seulement SQL ». Il a été toutefois popularisé à la suite de la rencontre de San Francisco de Juin 2009 où furent présentés les concepts de bases de données distribuées et libres. Ces bases s'inspirent des solutions créées par les grands acteurs de l'Internet que sont : Google, Amazon, Yahoo, ou Facebook. Elles sont utilisées par de nombreuses Startups et entreprises comme Twitter, Last.fm, LinkedIn ou encore Netflix.

De part leurs activités ces entreprises collectent et manipulent des quantités de données telles qu'elles ont besoin de solution de persistance performantes, capables de gérer des volumes considérables et bon marché. De ce fait on peut considérer que les projets de bases « NOSQL » sont issus de besoins non couverts par les systèmes de bases de données relationnelles courants.

Il est possible de représenter le positionnement des deux propositions : SGBD Relationnels et Bases « NoSQL » selon les domaines de performances de ces systèmes.

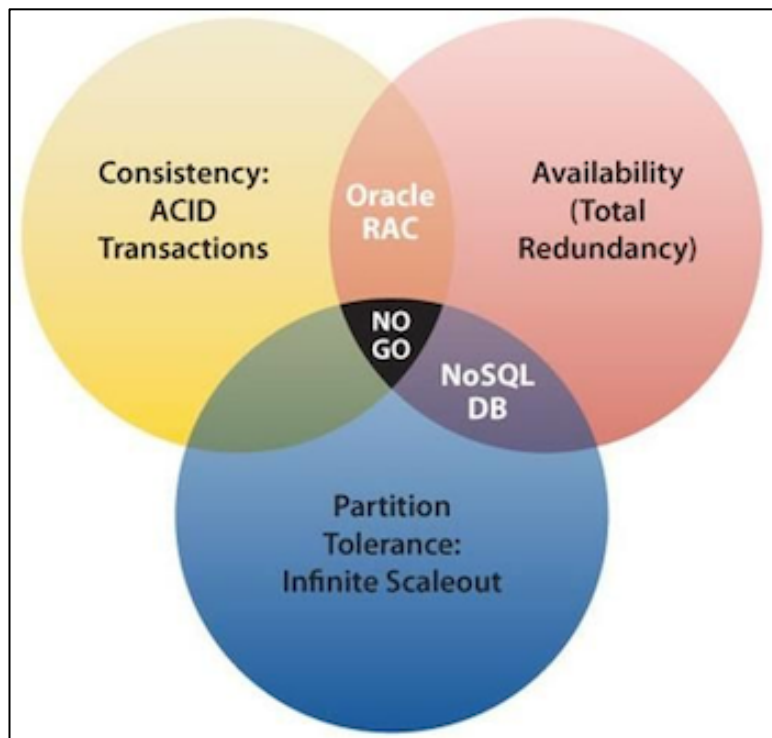


Figure 1 : SGBD Relationnels VS Bases NoSQL (<http://blog.inovia-conseil.fr>)

Ainsi et compte tenu du contexte le premier besoin auquel répond un système NoSQL est la performance et la scalabilité pour satisfaire une charge et volumétries croissantes de façon exponentielle. Ces solutions ont procédé à des compromis par rapport aux SGBD Relationnels pour offrir scalabilité horizontale et évolutivité. Contrairement aux SGBD Relationnels les systèmes NoSQL ne satisfont pas aux propriétés ACID (Atomicité, Cohérence, Isolation, Durabilité) et abandonnent le mode transactionnel et le langage de requête.

En revanche ces systèmes répondent au principes du théorème CAP (Consistency, Availability, Partition tolerance) ou théorème de Brewer. Ce théorème stipule qu'un système distribué ne peut garantir simultanément que deux des trois principes suivants :

- **La cohérence** : Tous les nœuds du système voient les mêmes données au même moment, la modification d'une donnée prend effet immédiatement,
- **La haute disponibilité** : La garantie que toutes les requêtes reçoivent une réponse qu'elle soit un succès ou un échec, en cas de panne les données restent accessibles,
- **La tolérance au partitionnement** : Le système peut être partitionné, et garantir la continuité du fonctionnement en cas d'ajout ou de suppression de nœud dans le système distribué.

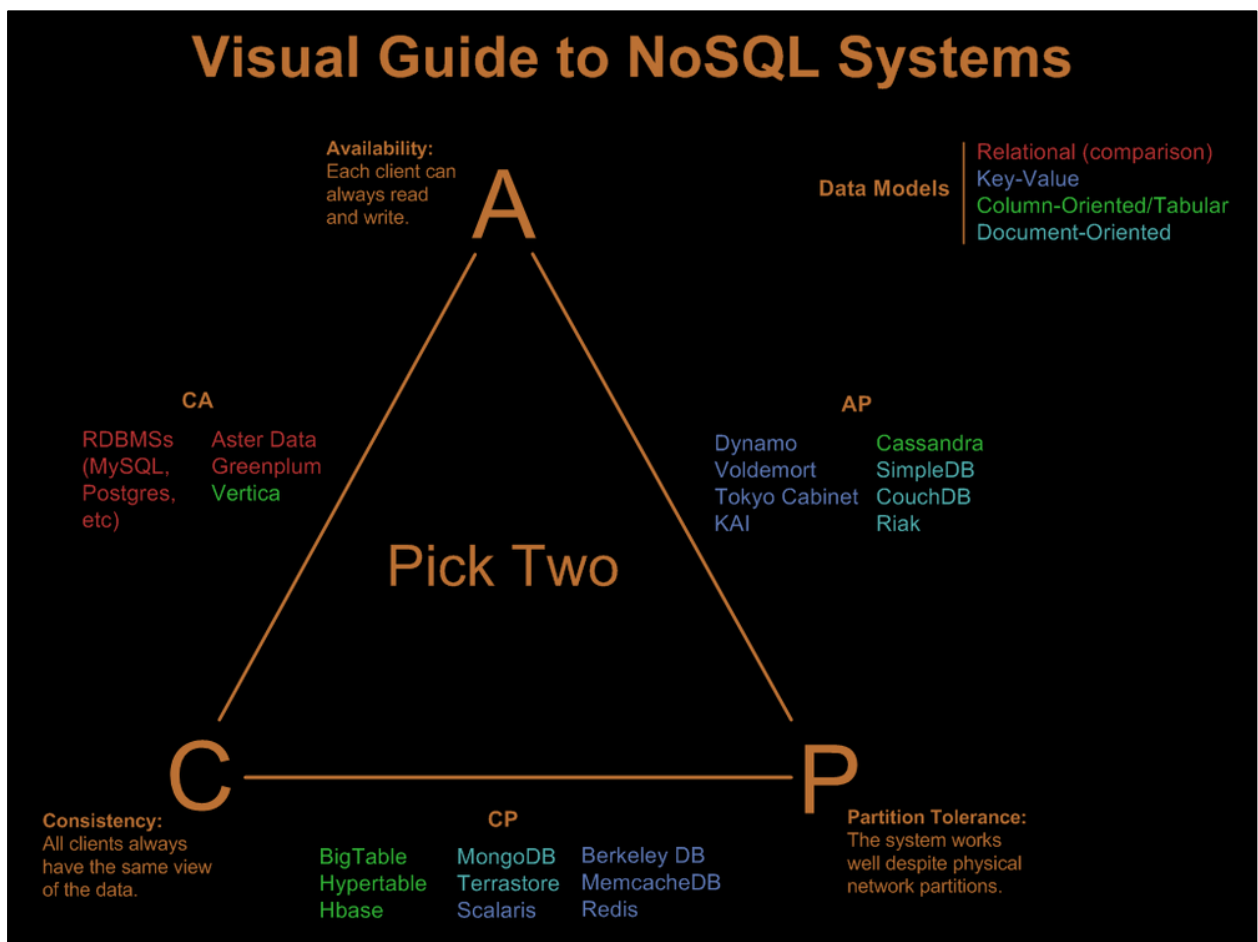


Figure 2 : Schématisation Théorème CAP (<http://blog.beany.co.kr/archives/275>)

Il existe d'autres caractéristiques communes aux bases de données « NoSQL » comme le partitionnement horizontal sur plusieurs nœuds, la réplication des données, des schémas dynamiques ou l'absence de schéma. Les bases de données « NoSQL » ont la particularité d'être hautement disponibles et tolérantes aux pannes et au partitionnement. Mais en général elles ne garantissent pas la cohérence stricte des données.

De très nombreuses solutions de bases « NoSQL » existent. Elles mettent en œuvre des techniques différentes pour couvrir le paradigme « CAP » et sont développées dans le but de pouvoir demeurer performantes tout en traitant un très grand volume de données et capable de gérer le passage à l'échelle par scalabilité horizontale.

Toutes ces propositions ne répondent pas aux mêmes besoins, on distingue ainsi 4 grandes familles de bases « NoSQL » :

1. Le modèle Clé / Valeur

Ce modèle de base de données peu être considéré comme une très grande table associative, une sorte de hashmap distribuée. Les données sont représentées par des couples (clé/valeur). La valeur peu être de différents types, de la simple chaine de caractères à l'objet sérialisé. Le modèle le plus proche est la mémoire cache.

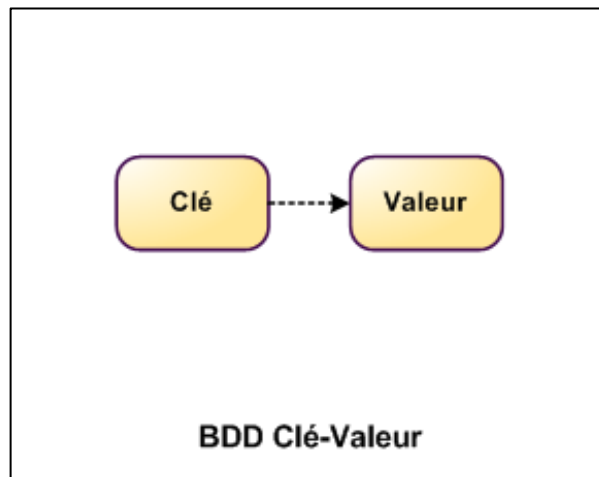


Figure 3 : Modèle Clé/Valeur (<http://blog.xebia.fr>)

Les structures de données ne sont pas assujetties à un schéma et ne possèdent pas de typage. En fonction des choix d'implémentation les clés pourront être attribuées ou générées aléatoirement au moment de l'ajout d'un enregistrement.

Le manque de schéma et de structure limite les opérations en base aux commandes « PUT », « GET » et « DELETE » et l'absence de langage de requête comme SQL contraint à exporter l'intelligence de traitement dans les applicatifs.

Ces bases de données peuvent être partitionnées sur plusieurs instances. Elles sont réputées pour leurs performances et leur capacité à traiter un grand volume de données.



Figure 4 : Modèle Clé/Valeur Distribué (<http://blog.inovia-conseil.fr>)

Elles sont bien adaptées pour :

- La collecte de logs (événements de jeux en ligne),
- La gestion des traces (mesures d'audiences),
- La gestion de profils d'utilisateurs (commerce électronique, sites à forte audience).

2. Le modèle Orienté colonne

Ce modèle de base de données s'apparente à une table d'une base de données relationnelle à la différence que le nombre de colonnes est dynamique. Le nombre de colonnes peut varier d'un enregistrement à l'autre. Ce principe évite les colonnes vides.

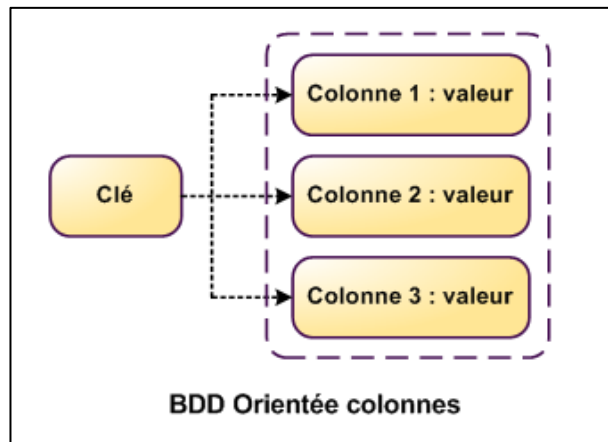


Figure 5 : Modèle orienté Colonnes (<http://blog.xebia.fr>)

Le dynamisme des colonnes peut être illustré comme suit dans le cadre d'un carnet d'adresses :

1	Nom	Fixe
	Patrick	0102030405
2	Nom	Mobile
	Farid	0601020304
3	Nom	
	Tim	

Figure 6 : Colonnes dynamiques (<http://blog.xebia.fr>)

Le modèle orienté colonnes est principalement mis en œuvre dans les solutions de « Google » avec « BigTable », ou « HBase » et « Cassandra » deux projets de la fondation « Apache ».

3. Le modèle orienté document :

Ce modèle de base de données s'appuie sur le principe clé/valeur, où la valeur est un document. Ces bases permettent de stocker des données semi-structurées, c'est-à-dire dont la structure n'est pas contrainte par un schéma mais dont le contenu respecte toutefois un certain format. L'unité de stockage est un document, généralement au format JSON ou XML.



Figure 7 : Modèle orienté Document (<http://blog.inovia-conseil.fr>)

D'une certaine façon, on peut définir un document comme un ensemble de couples propriété/valeur. On peut donc considérer la base de donnée orientée document comme une sorte d'évolution de la base de donnée clé/valeur distribuée.

Un document contient des données organisées de manière hiérarchique à l'image de ce que permettent XML ou JSON. La base connaît le contenu qu'elle stocke, il est alors possible d'effectuer des indexations de différents champs et offrir des requêtes élaborées.

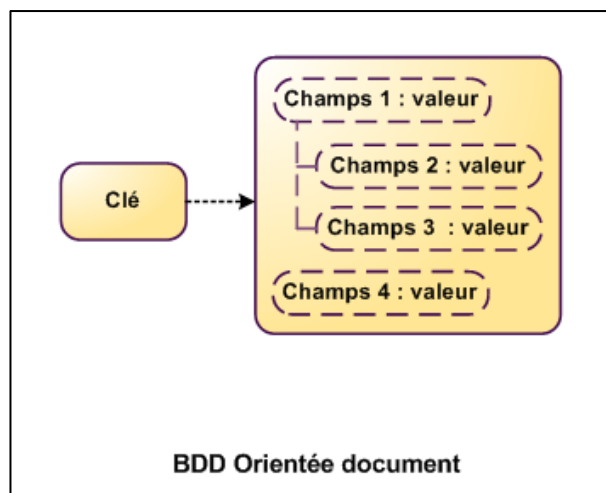


Figure 8 : Modèle orienté Document, représentation (<http://blog.xebia.fr>)

La représentation en document est particulièrement adaptée au monde du Web.

Ce type de base se rencontre dans les systèmes de gestion documentaire (CMS) car le modèle s'y prête bien mais est également adapté dans les cas d'usage suivants :

- Archivage / Stockage de traces applicatives,
- Statistiques et Analyse temps-réel,
- Plateforme de commerce électronique (souvent en association avec un SGBDR),
- Prototypage.

Les implémentations les plus courantes sont CouchDB et MongoDB.

4. Le modèle orienté Graphe :

Le modèle orienté Graphe se base sur la théorie des graphes pour représenter les données. Il s'appuie sur la notion de nœud, de relations et de propriétés qui leur sont rattachées. Les propriétés portent aussi bien sur les nœuds que les relations.

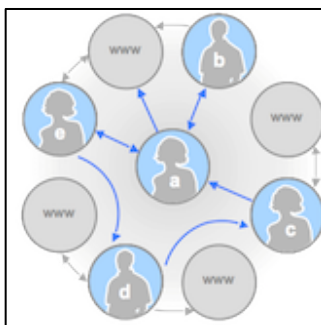


Figure 9 : Modèle Orienté Graphe (<http://blog.inovia-conseil.fr>)

Les bases de données orientées Graphes sont conçues pour modéliser des structures de données relationnelles aussi diverses qu'un réseau social, un réseau ferroviaire ou l'organisation d'une entreprise.

Les bases de données relationnelles répondent très bien à ce genre de problématique. Mais elles atteignent leurs limites en terme de performance quand la profondeur du graphe et le volume de données à gérer sont importants. Sur ce point, les bases de données orientées graphe surclassent les SGBDRs.

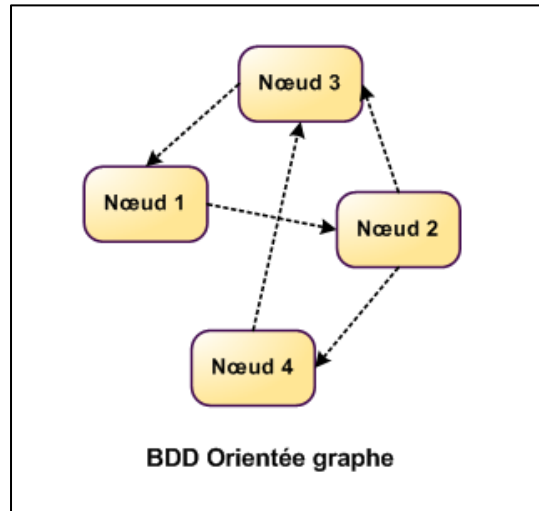


Figure 10 : Représentation Modèle orienté Graphe (<http://blog.xebia.fr>)

La représentation en graphe permet la modélisation, le stockage et la manipulation de données complexes liées par des relations non-triviales ou variables.

Ce modèle facilite la représentation du monde réel, ce qui le rend adapté au traitement des données des réseaux sociaux ou pour gérer des données spatiales.

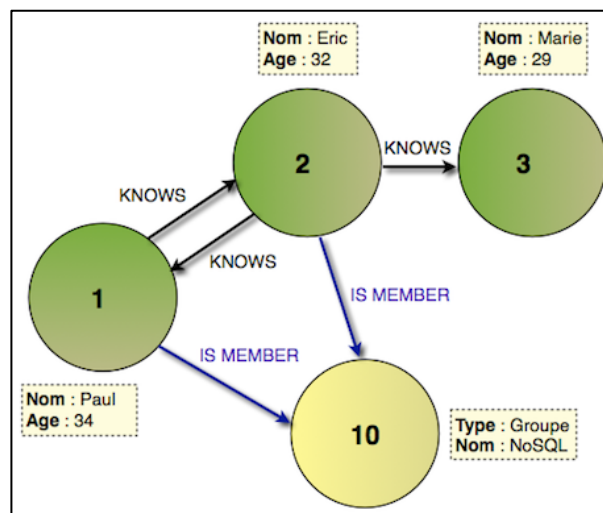


Figure 11 : Représentation sociale, modèle Graphe (<http://blog.xebia.fr>)

Ce modèle convient également pour représenter des informations financières où les relations modéliseraient les dépôts ou retraits.

Les implémentations les plus courantes sont Neo4j, HypergraphDB et FlockDB.

En conclusion ces quatre modèles ont des approches différentes pour traiter des données non structurées au sens Modèle relationnel, et s'efforcent de favoriser la performance lors du passage à l'échelle. Les deux mouvements "orienté colonne" et "orienté document" découlent du système clé valeur et c'est la nature ou la structure de la valeur qui diffère.

C'est l'approche qu'a choisie « AMAZON » lors du développement de la base « NoSQL » « DYNAMO » qu'ils utilisent largement dans leur offre « CLOUD »

1.2 L'approche d'AMAZON

En 2007 AMAZON a publié, par le biais de son directeur technique, un document sur le système de base NoSQL en gestation : DYNAMO. Cette base de donnée doit prendre en charge la gestion des paniers d'achat des clients d'AMAZON.

La société AMAZON a fait le constat que la fiabilité à très grande échelle est un des défis majeurs auquel est confronté le site marchand AMAZON.COM. Ce site de commerce fournit des services pour de très nombreux sites web à travers le monde. Il est installé sur une infrastructure comprenant des dizaines de milliers de serveurs répartis dans de très nombreux centres de données de part le monde.

A cette échelle les pannes sont continues et peuvent mettre en péril l'activité de l'entreprise en bloquant les services fournis.

Pour faire face à cela AMAZON a développé des techniques de stockage dont « DYNAMO » : un entrepôt de données distribué, hautement disponible et supportant le passage à l'échelle.

« DYNAMO » a été conçu pour assurer de bonnes performances pour un coût réduit en utilisant un ensemble de techniques courantes pour garantir la disponibilité, la scalabilité et la consistance.

Les données sont réparties et répliquées selon la méthode du « Consistent Hashing », la consistance est assurée en utilisant le « Object Versioning ». La cohérence entre réplicas lors de mises à jour est gérée par un ensemble de techniques et un protocole de synchronisation décentralisé s'appuyant sur la détection d'erreurs.

De très nombreux services d'AMAZON n'ont seulement besoin d'une clé primaire pour accéder aux données. Ce sont des services comme les préférences utilisateur, la gestion de session, le classement des ventes, le catalogue de produits ou une liste de meilleures ventes. Pour ces besoins l'utilisation d'une base de données relationnelle serait inefficace et poserait des limites d'échelle et de disponibilité. C'est pour cela qu'AMAZON a conçu une base de données s'appuyant sur le simple principe de la clé primaire pour répondre aux besoins de ces services.

1.3 La base DYNAMO

Plus explicitement, le cahier des charges de « DYNAMO » se résume en trois principaux principes :

1. Un modèle d'interrogation simple : Des opérations de lecture et d'écritures basées seulement sur une clé et en accès direct. De fait il n'est pas besoin de schéma relationnel.
2. Abandon des propriétés ACID qui garantissent la fiabilité des transactions dans les bases relationnelles. Ces propriétés vont à l'encontre de la disponibilité. DYNAMO fournit

donc une moindre cohérence pour permettre une très haute disponibilité. De plus DYNAMO ne garanti pas l'isolation et n'autorise pas les mises à jour sur des clés multiples,

3. Efficacité : Le système doit fonctionner sur une infrastructure matérielle hétérogène. Les services doivent pouvoir configurer DYNAMO de sorte qu'ils respectent une latence commune.

De plus il n'est pas prévu de système de sécurité spécifique comme l'authentification, le système DYNAMO est censé être opéré en interne par AMAZON sur dans un environnement système fiable. Chaque service utilisant une instance séparée de DYNAMO, il est prévu de gérer des centaines de points de stockage.

L'engagement de service de DYNAMO est de fournir une réponse en 300ms pour 99,9% des requêtes lors d'un pic de connexion de 500 requêtes par secondes.

DYNAMO s'inscrit dans une architecture de service qui peut être représentée de la façon suivante dans l'environnement AMAZON

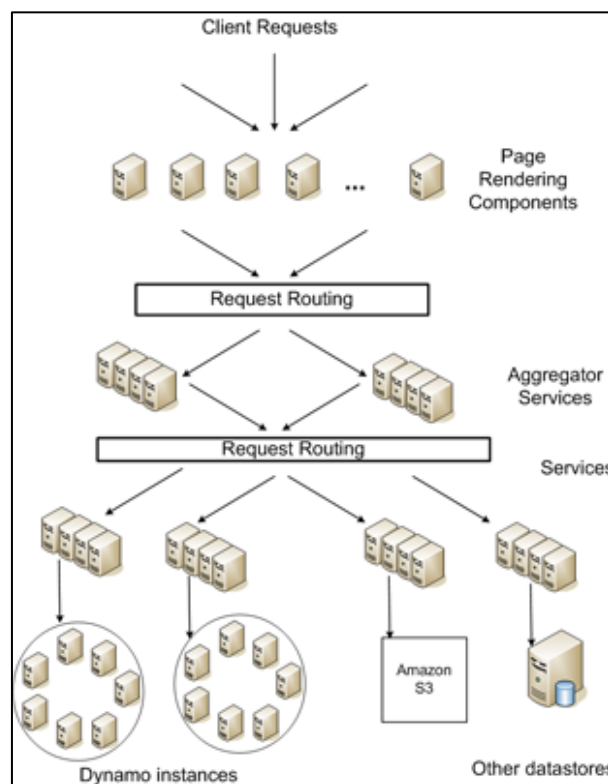


Figure 12 : Architecture AMAZON et DYNAMO (Article Amazon¹)

Il n'est pas possible dans une base de données répliquée d'assumer en même temps la gestion des erreurs réseau, une forte cohérence et une haute disponibilité. De ce fait DYNAMO garantit une cohérence à terme, DYNAMO permet les opérations de lecture et d'écriture pendant les phases de partitionnement réseau.

¹ Article « Dynamo: Amazon's Highly Available Key-value Store »

2 LE PROJET VOLDEMORT

2.1 Historique

Un grand nom du web pour ses besoins internes s'est penché sur les systèmes NoSQL.

LinkedIn un acteur majeur dans le monde des réseaux sociaux professionnels a des besoins spécifiques pour son activité.

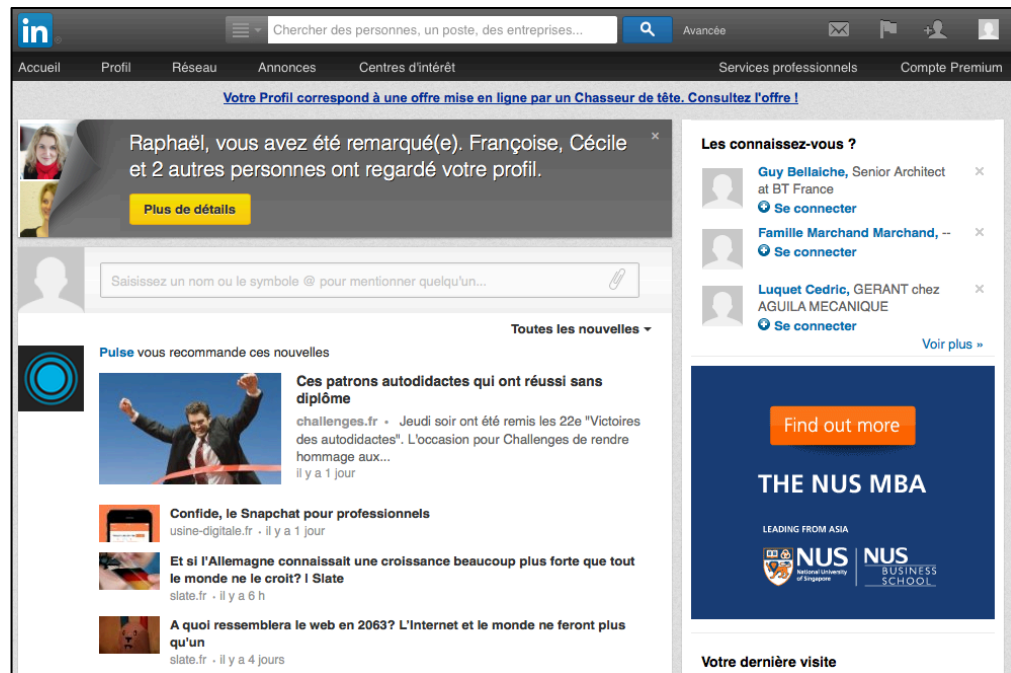


Figure 13 : LinkedIn page contacts (LinkedIn.fr)

LinkedIn en tant que grand utilisateur de systèmes libres, utilisait et participait déjà au développement d'outils comme « Lucene », « Zoie », « Kamikaze » ou « Bobo ». La plupart de ces contributions étaient très spécifiques à LinkedIn. Mais il manquait des briques pour la recherche et le stockage des données en complément de Hadoop. Les services de recherches de LinkedIn : « Data & Analytics » et « Analysis & Research » se sont inspiré de la solution proposée par AMAZON pour concevoir leur base de donnée Clé-Valeur.

LINKEDIN a fait ce choix pour répondre aux besoins de recherche et de représentation des relations entre membres. C'est un modèle en rupture avec les bases de données relationnelles. En effet il n'est pas judicieux en relationnel d'assembler des chaînes pour atteindre les données. Il est compliqué de construire des structures souples et réutilisables. Le modèle dont a besoin LinkedIn s'appuie sur les services sans réel besoin de jointure. LinkedIn a en revanche besoin d'un modèle dé normalisé, avec une API simple et des accès aux données utilisant de simples commandes GET. L'intelligence se trouve déportée dans l'application.

Comme l'architecture de LinkedIn a besoin de scalabilité horizontale et que le facteur clé est la latence : Le choix du NoSQL est tout indiqué.

Quelques besoins métiers pour lesquels l'approche NoSQL est envisagée :

- Personnes que vous pourriez connaître,
- Recommandations d'éléments,

- Données dérivées de membres et d'entreprises,
- Statistiques du réseau,
- Qui a consulté mon profil ?
- Des données traitées en Batch et présentées en lecture seulement,

Quelques contraintes auxquelles doit faire face la solution :

- Les jeux de données sont volumineux et persistants,
- Tout ne peut être stocké en mémoire,
- Les données doivent être distribuées sur plusieurs machines,
- Besoin de contrôle sur la disponibilité et la durabilité des données,
- Il faut répliquer les données sur de multiples serveurs,
- Le coût du passage à l'échelle ne doit pas être trop important,
- Doit supporter de multiples utilisations.

LinkedIn c'est donc appuyé sur les travaux de AMAZON et notamment sur un article publié à la conférence SOSP de 2007 décrivant l'entrepôt de données utilisé pour la gestion des paniers clients.

LinkedIn développe alors une base de donnée : « VOLDEMORT » du nom d'un personnage d'un célèbre roman. Fidèle à sa philosophie LinkedIn Proposera « VOLDEMORT » sous licence libre Apache 2.0 dès 2009.

2.2 ***VOLDEMORT en détails***

VOLDEMORT est une base de données de type clé/valeur dans laquelle les données sont :

- Automatiquement répliquées sur de nombreux serveurs,
- Automatiquement partitionnée de sorte que chaque serveur ne contienne qu'un sous ensemble des données,
- Permet une cohérence ajustable (cohérence à terme ou cohérence stricte),
- La défaillance d'un serveur est gérée de façon transparente,
- Utilise différents types de moteurs de stockage (BDB, MySQL, in Memory, Read Only),
- Les données sont versionnées pour augmenter l'intégrité sans compromettre la disponibilité en cas panne.
- Chaque nœud est indépendant des autres, pas de coordination ni de point central de faiblesse.
- Supporte des stratégies de placement géographique des données pour assurer la distribution au travers de centres de données éloignés.

VOLDEMORT est donc utilisé dans des cas où le simple partitionnement des données n'est pas suffisant. Ce n'est pas une base de donnée relationnelle satisfaisant aux propriétés ACID. VOLDEMORT ne propose pas de nouveau concept comme un modèle orienté document, VOLDEMORT se présente comme une très grande table de hachage distribuée, persistante et résistante aux pannes.

Il s'applique aux applications internet qui ont besoin de scalabilité partageant leurs services ou API à travers de multiples centres de données utilisant des systèmes de stockage pouvant eux même être partitionnés horizontalement.

2.3 Les principes mis en œuvre

2.3.1 Une architecture en couches :

VOLDEMORT propose une API JAVA simple qui permet d'effectuer les trois commandes de base : Get, Put et Delete. Chaque couche prend en charge une fonction simple comme la communication réseau sur TCP/IP, la sérialisation, la résolution de conflit grâce à la gestion de version ou encore le routage entre les nœuds.

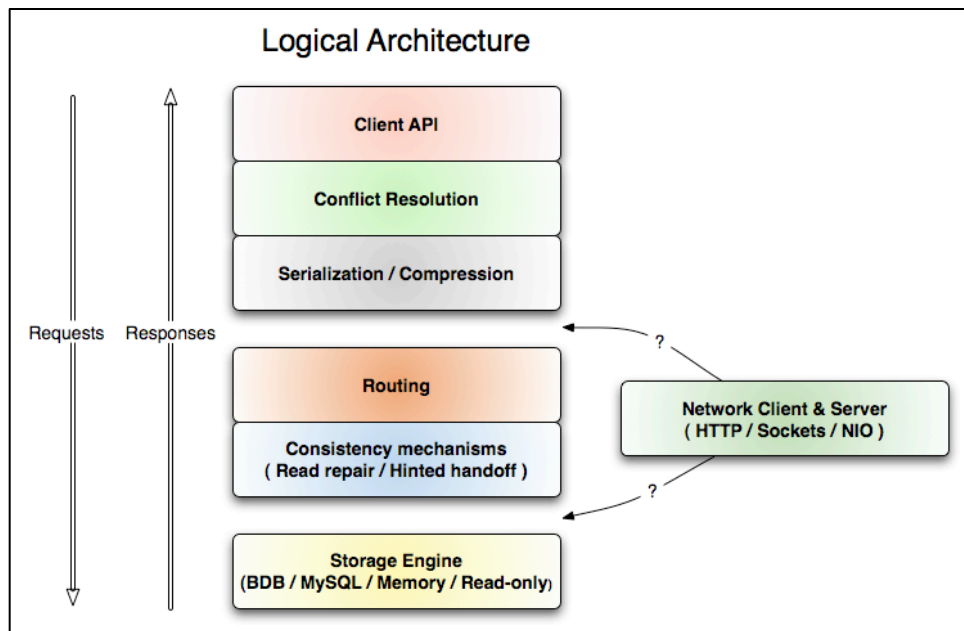


Figure 14 : Architecture VOLDEMORT (<http://www.project-voldemort.com/voldemort/>)

Cette architecture est très flexible, chaque couche étant responsable de fonctions simples, il est possible d'en ajouter de nouvelles, ou de les mixer à l'exécution selon les besoins. Ce découpage permet en outre faciliter les tests.

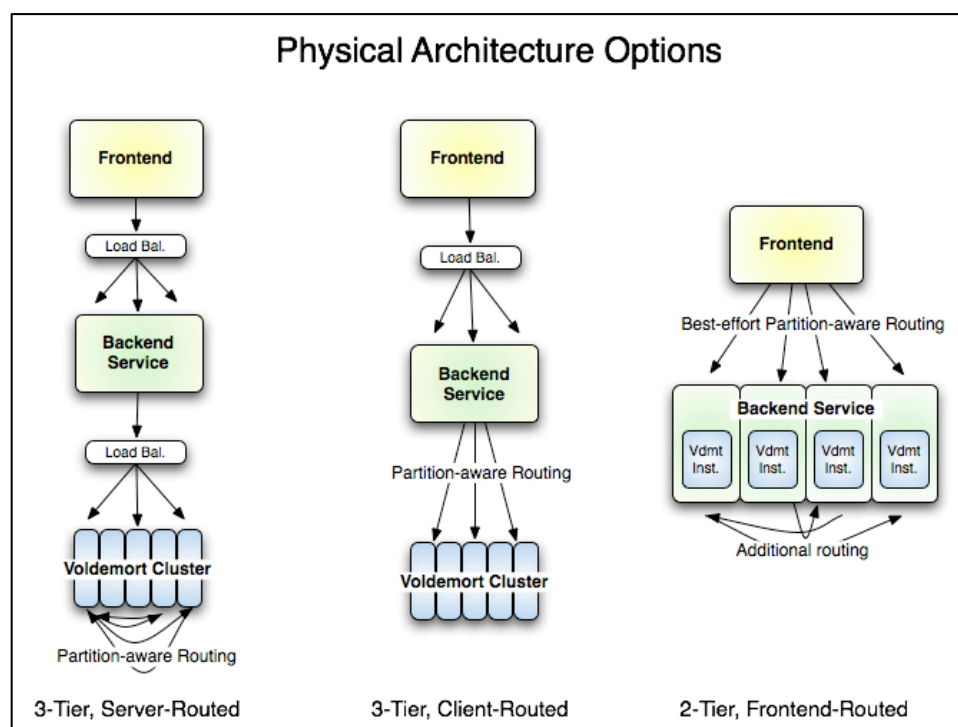


Figure 15 : Architecture Physique VOLDEMORT (<http://www.project-voldemort.com/voldemort/>)

Une architecture optimale pour la performance consiste à combiner un répartiteur de charge entre la partie applicative et la partie entrepôt de données. De la sorte les requêtes seront routées au plus près du serveur hébergeant les données cible.

Cette flexibilité permet des configurations performantes en réduisant le nombre de redirections réseau pour router le service directement sur le bon serveur. Sur la Figure 15 la représentation de 3 configurations implémentant 1, 2 ou trois redirections.

2.3.2 La disponibilité

VOLDEMORT en tant que système NoSQL est contraint par le théorème de CAP, LinkedIn a donc fait le choix de la disponibilité et de la performance devant la cohérence mais sans négliger l'intégrité des données.

La disponibilité est assurée selon ces principes :

- un mode master/master : Tous les nœuds sont disponibles en écriture. Un mécanisme de répartition de charge permet de dispatcher l'ensemble des requêtes sur l'ensemble des nœuds. L'utilisation de l'algorithme Vector Clock permet de suivre et de conserver les différentes versions ainsi que l'historique des objets notamment dans le cas d'écriture sur différents nœuds « master ». Des échanges protocolaires (type « gossip protocol ») permettent de propager de l'information (serveur perdu)
- Partitionnement et réplication automatique des données entre les différents serveurs qui composent le cluster de base avec des enjeux d'optimisation des coûts (« commodity hardware ») et de résilience des systèmes. Le consistent hash est utilisé pour la répartition des clés sur les nœuds. Cet algorithme permet de limiter la quantité de données à re distribuer sur les nœuds (pour respecter les configurations de réplication, de consistance).
- Un modèle consistant à terme : Une alternative à ACID et au Two Phase Commit. Dans cette approche, toutes les versions des objets sont écrites (l'historique est géré par le « Vector Clock »), les conflits éventuels sont détectés et gérés au moment de la lecture de l'objet : mode « Read-Repair ». La réconciliation des objets se fait à la lecture en se basant sur les informations contenues dans le « Vector Clock ». La consistance quand à elle est garantie par le nombre de serveurs possédant la même version de cette donnée. La consistance peut être paramétrée, c'est-à-dire qu'il est possible de définir, type de données par type de données, le nombre de nœuds sur lesquels une donnée doit être écrite pour considérer une écriture valide ou bien le nombre de nœuds sur lesquels une donnée doit être lue pour être considérée comme consistante.

2.3.3 La distribution, Consistent Hashing

Les données ne pouvant être stockées sur un seul serveur, elles doivent être distribuées au travers d'un cluster de serveurs. Pour de bonnes performances ceci est fait même lorsque les données peuvent être entièrement hébergées sur un seul serveur (scalabilité horizontale). Ceci a pour effet d'améliorer l'efficacité des caches serveur en permettant à des petits jeux de données d'être complètement stockés en mémoire.

Le mécanisme se base sur une fonction de haching appliquée sur la clé pour sélectionner l'instance (le serveur) qui se chargera du stockage. Si la fonction de hachage utilisée est uniforme, les données seront équitablement réparties.

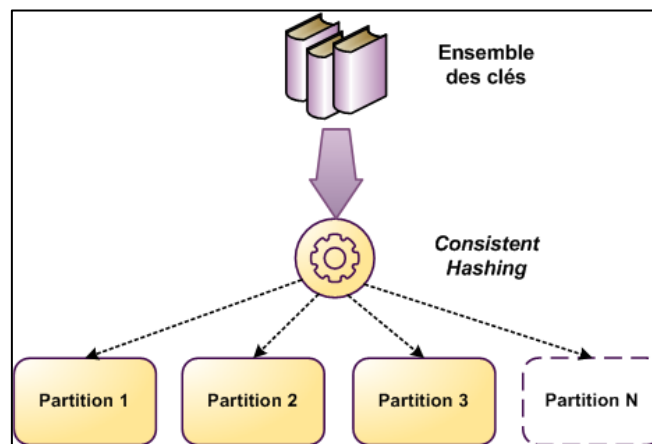


Figure 16 : Consistent Hashing (<http://blog.xebia.fr/>)

L'ensemble des clés est subdivisé en partitions, chaque partition est assignée à un nœud (Serveur). Un serveur peut héberger plusieurs partitions en fonction de ses capacités.

Du fait de la possibilité de déterminer la partition à laquelle appartient une donnée en connaissant uniquement sa clé il est possible de s'affranchir de l'utilisation d'un routeur central qui hébergerait le répertoire complet. De fait le routage peut se faire en point à point entre les serveurs du cluster.

Cette approche permet d'éviter les problèmes de redistribution complète des clés lors d'un ajout ou d'une suppression de serveur du cluster. Par exemple, lors de la défaillance d'un serveur VOLDEMORT va répartir la charge sur les serveurs restant dans le cluster. Lorsqu'un nouveau serveur est ajouté il ne sera nécessaire de redistribuer qu'une fraction des clés ($1/(\text{nombre de serveurs} + 1)$) sur le nouveau serveur.

2.3.4 Structure de données et sérialisation

VOLDEMORT stocke les données en les regroupant par type dans des « DATA Stores », une sorte d'annuaire par analogie à une table dans le monde relationnel. Chaque clé est unique dans un store et peut avoir au plus une valeur.

Le datastore détermine le type de donnée attendue pour la clé ainsi que le type de donnée pour la valeur associée à la clé.

Le « DATASTORE » définit ainsi une sorte de schéma pour les données, ce qui rend la sérialisation flexible voir même de la personnaliser en déterminant son propre modèle.

Les types suivant sont supportés de base dans la configuration des DataStores :

- JSON,
- Chaîne de caractère (String),
- Java Sérialisation d'objets,
- Protobuf : Format Google, protocole de sérialisation de buffers,
- Avro-Generic / Avro-Specific / Avro-Reflective : Un autre système riche de sérialisation,

Il est possible d'affecter un schéma (descriptif de donnée) pour la clé et pour la valeur de façon indépendante dans un store.

Le format JSON est un bon modèle de données car il supporte les types et la structure utilisés dans la plupart des langage de programmation (strings, numbers, lists/arrays, et objects/hashtables). Il permet d'utiliser les types suivants :

- int8, int16, int32, int64, float32, float64,
- string, date, object, bytes, boolean, object, array

Ci dessous une liste complète des types autorisés :

type	storable substyles	bytes used	Java type	example JSON	example type definition
number	int8, int16, int32, int64, float32, float64, date	8, 16, 32, 64, 32, 64, 32	Byte, Short, Integer, Long, Float, Double, Date	1	"int32"
string	string, bytes	2 + length of string or bytes	String, byte[]	"hello"	"string"
boolean	boolean	1	Boolean	true	"boolean"
object	object	1 + size of contents	Map<String, Object>	{"key1": 1, "key2": "2", "key3": false}	{"name": "string", "height": "int16"}
array	array	size * sizeof(type)	List<?>	[1, 2, 3]	["int32"]

Figure 17 : Types de données pour schéma

Un exemple simple de déclaration de données dans un « DataStore » donnerait :

```
<key-serializer>
  <type>string</type>
</key-serializer>
<value-serializer>
  <type>string</type>
</value-serializer>
```

Ou encore dans le cas d'une déclaration de type JSON :

```
{"fname": "string", "lname": "string", "id": "int32", "emails": ["string"]}
```

2.3.5 La réplication et la consistance

VOLDEMORT assure une cohérence, à terme, des données. C'est à dire qu'il est admis qu'à un instant précis plusieurs versions d'une donnée peuvent cohabiter dans le système. Mais au

moment de la lecture (Fonction Get) l'algorithme basé sur les « vector clock » gèrera les conflits. Ceci implique que les données sont répliquées sur plusieurs serveurs faisant parti d'une ferme (un cluster).

En exploitation quotidienne des serveurs tombent régulièrement en panne, sont surchargés ou sont indisponibles pour des opérations de maintenance. Pour ces raisons et afin de ne pas interrompre la production il faut stocker des informations unitaires sur un seul serveur. La solution la plus simple est de subdiviser les informations en plusieurs partitions (répartition de clés en partitions) et de répliquer ces partitions sur plusieurs serveurs.

Lors de la configuration du cluster de serveurs il faut alors définir le nombre de nœuds sur lequel la donnée doit être répliquée pour considérer une écriture valide (W) ainsi les réplicas seront placés sur $W-1$ autres nœuds. Si $W=3$ alors une clé (partition P) sur le noeud S sera répliquée sur les noeuds $S-1$ et $S+1$.

Un second facteur sera le nombre de nœuds sur lequel la donnée doit être lue (R) pour être considérée comme consistante (Lecture consistante si lue à l'identique sur R nœuds).

Le modèle sera alors considéré comme consistant s'il respecte la formule : $W+R > N$, N étant le nombre total de nœuds sur lesquels la donnée est répliquée. Une lecture suivant immédiatement une écriture est garantie de retourner la dernière valeur écrite. Cette configuration se fait au niveau de chacun des « Store ».

Exemple de définition de cluster de base

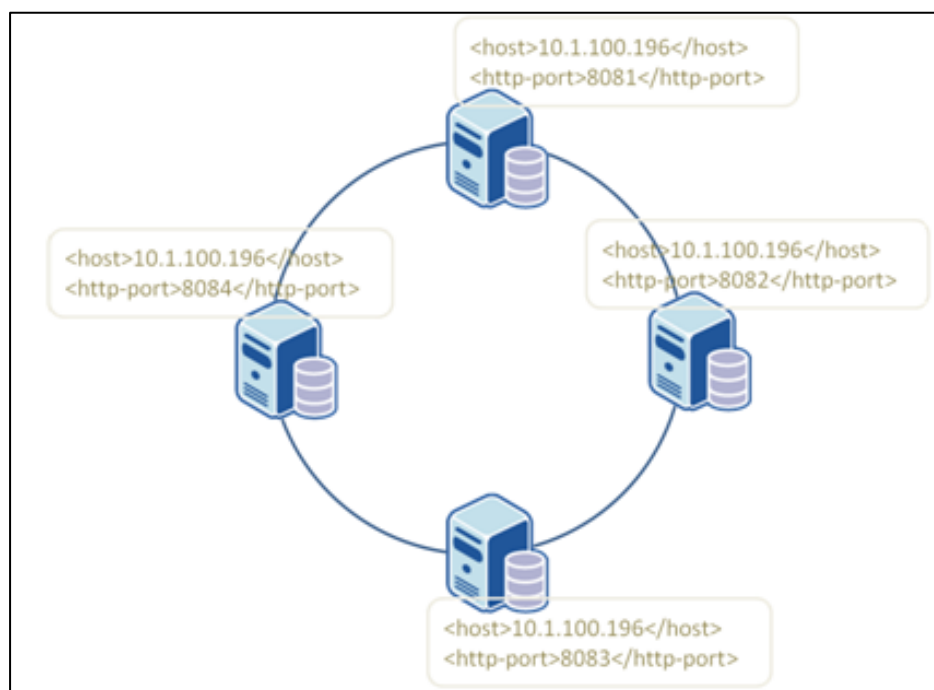


Figure 18 : Cluster configuration (<http://blog.octo.com>)

2.3.6 Les versions de données

Lorsque deux écriture successives concernent des valeurs différentes d'une donnée la donnée n'est pas signalée incohérente, on admet dans le modèle NoSQL la possibilité d'avoir plusieurs valeur de d'une donnée à un moment donné. La résolution de la cohérence à terme se faisant au moment de la lecture. VOLDEMORT associe donc un numéro de version à chaque donnée en utilisant la méthode des vector clocks. Il s'agit d'un compteur pour chaque serveur, ce compteur est associé aux écritures sur le serveur qui permet de déterminer quand deux versions entrent en conflit en déterminant simplement l'ordre d'écriture des versions concurrentes.

Un vector clock se présente sous la forme d'associations [Serveur :Version] (i.e. [1:45,2:3,5:55]) pour chaque clé.

Il sera ainsi aisé de résoudre un conflit de version lors de la lecture en comparant les versions des clés lues sur les différents serveurs afin d'assurer le nombre de lectures identiques choisies pour définir la cohérence.

2.4 Simplicité d'installation et de configuration

VOLDEMORT est disponible au téléchargement sur le site officiel : <http://www.project-voldemort.com/voldemort/> dans la rubrique « Download Latest ». Il est proposé sous la forme d'un fichier d'archive qu'il suffit simplement de décompresser à l'endroit souhaité.

Un répertoire « /bin » contient les scripts et exécutables pour les serveur, clients et autres outils d'administration.

La configuration du système est des plus aisée ; Elle consiste en trois fichiers de configuration qui doivent être présent sur chaque serveur du cluster.

Ces fichiers sont :

1. Serveur.properties : Sert à l'identification des nœuds, il contient l'identifiant du nœud ainsi que quelques paramètres locaux comme le nombre de threads simultanés autorisés ou encore le type de stockage local (i.e. BDB, MYSQL...)

```
# The ID of this particular cluster node
node.id=0
max.threads=100
enable.repair=false
##### DB options #####
# BDB
bdb.cache.size=100MB
```

Figure 19 : Identification du Nœud

2. Cluster.XML : Définit la configuration du cluster ainsi que la répartition des partitions (distribution des clés) sur les nœuds.

```
<cluster>
  <name>mycluster</name>
  <server>
    <id>0</id>
    <host>localhost</host>
    <http-port>8081</http-port>
    <socket-port>6666</socket-port>
    <admin-port>6667</admin-port>
    <partitions>0, 1</partitions>
  </server>
  <server>
    <id>1</id>
    <host>localhost</host>
    <http-port>8082</http-port>
    <socket-port>6668</socket-port>
    <admin-port>6669</admin-port>
    <partitions>2, 3</partitions>
  </server>
  <server>
    <id>2</id>
    <host>localhost</host>
    <http-port>8083</http-port>
    <socket-port>6670</socket-port>
    <admin-port>6671</admin-port>
    <partitions>4, 5</partitions>
  </server>
  <id>3</id>
  <host>localhost</host>
  <http-port>8084</http-port>
  <socket-port>6672</socket-port>
  <admin-port>6673</admin-port>
  <partitions>6</partitions>
</server>
</cluster>
```

Figure 20 : Définition du cluster

3. Store.XML : Dans ce fichier trois types d'informations sont définis
- a. Le facteur de réplication est configuré en précisant le nombre d'écritures nécessaire ainsi que le type de stockage à utiliser.
 - b. La cohérence par le biais des indicateurs du nombre d'écritures et de lectures nécessaire pour considérer une valeur de clé cohérente.
 - c. La structure de données stockées en précisant le type de la clé et de la valeur associée selon les types décrits au chapitre 2.3.4.

```
<stores>
  <!-- Note that "test" store requires 2 reads and 2
  writes,
  so to use this store you must have both nodes
  started and running -->
  <store>
    <name>test</name>
    <persistence>bdb</persistence>
    <routing>client</routing>

    <replication-factor>3</replication-factor>

    <required-reads>2</required-reads>
    <required-writes>2</required-writes>

    <key-serializer>
      <type>string</type>
    </key-serializer>
    <value-serializer>
      <type>string</type>
    </value-serializer>
  </store>
</stores>
```

Figure 21 : Réplication, cohérence et structure

3 DISCUSSION SUR L'APPROCHE NOSQL

VOLDEMORT comme DYNAMO propose un modèle consistant à terme. C'est à dire qu'il est structuré pour garantir de bonnes performances en écriture (il faut que le client puisse valider sa commande le plus rapidement possible, quelques soient les pannes) sur des systèmes soumis a de fortes charges, la réconciliation se fait au moment de la lecture (mode « Read-Repair »). Il y a deux « inconvénients » liés à cette approche. Le premier est la forte probabilité d'avoir plusieurs versions de la même valeur. Le second est de ne pas garantir qu'une lecture consécutive à la mise à jour d'une donnée retourne la donnée à jour (principe de la consistance à terme).

VOLEMORT propose un modèle moins structuré (non relationnel) qui présente d'autres inconvénients.

Au niveau des développements en premier lieu : Lorsque le modèle relationnel cherche à normaliser la donnée pour éviter la duplication et faciliter la gestion des relations entre entités, le modèle clé/valeur n'offre rien. Il n'y a pas de relations entre entités. De même il n'y a pas de langage de requête développé comme le SQL. Il faudra tout gérer au niveau de l'applicatif.

En terme d'API, le modèle clé/valeur est très simple, les seules APIs a disposition sont PUT, GET, DELETE. Toute l'intelligence remonte dans l'espace applicatif avec un fort impact sur les architectures logicielles.

Un autre point d'attention est la génération des clés. En effet, tout est clé mais ces systèmes ne mettent à disposition aucun mécanismes de génération de clé, là encore il faudra gérer au niveau applicatif.

Mais paradoxalement, ces contraintes sur le modèle de données ont des vertus sur les architectures applicatives. Tous les objets sont identifiables par une clé et les seules APIs disponibles sont PUT/GET/DELETE. Pas très loin de certains préceptes REST. Des contraintes qui finalement rendent les services très simples : Les signatures vont se limiter à un paramètre qui est l'identifiant de l'objet.

La simplicité du modèle de donnée et la sérialisation qu'il permet rapproche encore plus les données du développement, ce qui constitue un avantage certain.

CONCLUSION

Les systèmes « NOSQL » peuvent manipuler de très grandes quantités de données, offre une scalabilité importante pour répondre à la charge. Ils sont ainsi très adaptés aux besoins des applications de l'Internet. Ces produits sont des outils de stockage de données.

Le NoSQL ne vient pas en remplacement des Bases de données relationnelles mais se positionne en complément des SGBD relationnels pour fournir des solutions plus adaptées à certains contextes applicatifs. Typiquement les applications du WEB.

C'est pourquoi des solutions comme DYNAMO et VOLDEMORT sont issues de la recherche de grandes entreprises de l'Internet et sont de plus en plus déployées dans les parties des systèmes d'information en forte interaction avec le WEB.

De plus ces technologies suscitent l'engouement grâce à leur simplicité de mise en œuvre et à leur bon fonctionnement dans des conditions extrêmes qui leur confère une fiabilité déjà reconnue.

Quelques idées fausses à propos du NoSQL :

- Le NoSQL n'est pas une technologie de remplacement des SGBDR. Le NoSQL apporte des options supplémentaires quant au mode de représentation de données utilisé pour un projet.
- Le NoSQL est encore récent, les outils s'étoffent et les communautés bien que très actives, ne sont pas comparables à celles des SGBDR courants. Les produits ne sont pas encore au niveau de maturité de SGBDR comme MySQL ou PostgreSQL.
- L'intérêt d'une base de données NoSQL pour un projet ne dépend pas du volume de données qu'elle aura à manipuler. Le choix de son utilisation doit être basé sur la préférence d'un mode de représentation et la typologie des données plutôt que sur une forte volumétrie.
- Le NoSQL n'est pas une solution miracle pour tous les types de stockage de données. La tentative de reproduire dans une base de données NoSQL une représentation ou un comportement habituellement offert par un SGBDR aboutira probablement à une solution peu efficace.

Glossaire

Propriétés ACID : (atomicité, cohérence, isolation et durabilité) ; Ensemble de propriétés qui garantissent qu'une transaction informatique est exécutée de façon fiable.

VM (Virtual Machine) : Machine virtuelle.

SGBD : Système de gestion de base de données ; Logiciel système destiné à stocker et à partager des informations dans une base de données

BDB : Berkeley DB ; Moteur de base de données. Il est utilisé dans de nombreux environnements, sous Unix, GNU/Linux, Microsoft Windows et dans des systèmes embarqués

MYSQL : Un système de gestion de base de données. Il est distribué sous licence GPL.

HASHMAP : Table de hachage ; Structure de données qui permet une association clé-élément,

REST : REpresentational State Transfer ; Style d'architecture pour les systèmes hypermédia distribués

CAP : théorème énonçant qu'il est impossible sur un système informatique de calcul distribué de garantir en même temps les trois contraintes suivantes : Cohérence (tous les nœuds du système voient exactement les mêmes données au même moment), Disponibilité (Availability garantie que toutes les requêtes reçoivent une réponse et Résistance au morcellement

Webographie

Références relatives à Dynamo

AMAZON : Amazon Web Services

Liens :

<http://aws.amazon.com/fr/dynamodb/>, Présentation

<https://aws.amazon.com/fr/documentation/dynamodb/>, Documentation technique

Description détaillée d'Amazon SimpleDB : <http://aws.amazon.com/fr/simplifiedb/details/>

Article : « Dynamo: Amazon's Highly Available Key-value Store »

Lien :

<http://www.read.seas.harvard.edu/~kohler/class/cs239-w08/decandia07dynamo.pdf>

Références LinkedIn :

- **Project Voldemort** ; <http://data.linkedin.com/blog/tag/voldemort>
- **Project Voldemort (Part II): How it works** ;
<http://blog.linkedin.com/2009/04/01/project-voldemort-part-ii-how-it-works/>
- **Project Voldemort: Scaling Simple Storage at LinkedIn** ;
<http://blog.linkedin.com/2009/03/20/project-voldemort-scaling-simple-storage-at-linkedin/>

Présentation LinkedIn

- <http://behemoth.strlen.net/~alex/voldemort-devoxx.pdf>
- <http://www.odbms.org/download/voldemort.pdf>

Références au projet VOLDEMORT

Site : <http://www.project-voldemort.com>, dossier « Design »

Lien : <http://www.project-voldemort.com/voldemort/>

Blog Octo : Articles « SimpleDB le harry potter de Voldemort » et « Des alternatives aux bases de données relationnelles »

Liens :

<http://blog.octo.com/amazon-simplifiedb-le-harry-potter-de-voldemort/>

<http://blog.octo.com/des-alternatives-aux-bases-de-donnees-relationnelles.../>

Références diverses

Article : « NoSQL Europe : Tour d'horizon des bases de données NoSQL » par Michaël Figuière le 21 avril 2010. Catégorie(s) : NoSQL / Big Data

Lien :

<http://blog.xebia.fr/2010/04/21/nosql-europe-tour-dhorizon-des-bases-de-donnees-nosql/>

Article : « Panorama des bases de données NoSQL » par Kadda SAHNINE. Publié le 13 novembre 2012

Lien :

<http://blog.inovia-conseil.fr/?p=117>